

中国科学技术大学

# 蜗壳牌电子薯片

Walker's Electronic Chip

龙芯杯 2026

CPUSTCore 处理器设计报告

学校 中国科学技术大学

团队成员 赵天翔、刘亦航、董宇皓、薛浩聪

日期 2026 年 8 月

## CHAPTER 1

## 整体设计

## 1.1 系统总体架构

CPUSTCore 是一款基于 LA32R 乱序三发射的超标量处理器核心，采用基于显式物理寄存器重命名的现代乱序执行后端，通过集中式 Issue Queue 进行 Tomasulo 风格的 tag 唤醒与动态调度，并由 ROB 保证按序提交。核心实现整数、乘除、访存和系统指令，具备 CSR、精确异常、中断与 MMU/TLB，并通过 AXI 主接口接入片上系统，可作为 CPUSTC SoC 的主处理器运行 U-Boot 与 Linux。

核心采用每拍 4 条指令取指、3 宽译码与 3 宽提交。图 1.1 给出处理器的数据通路和控制闭环。

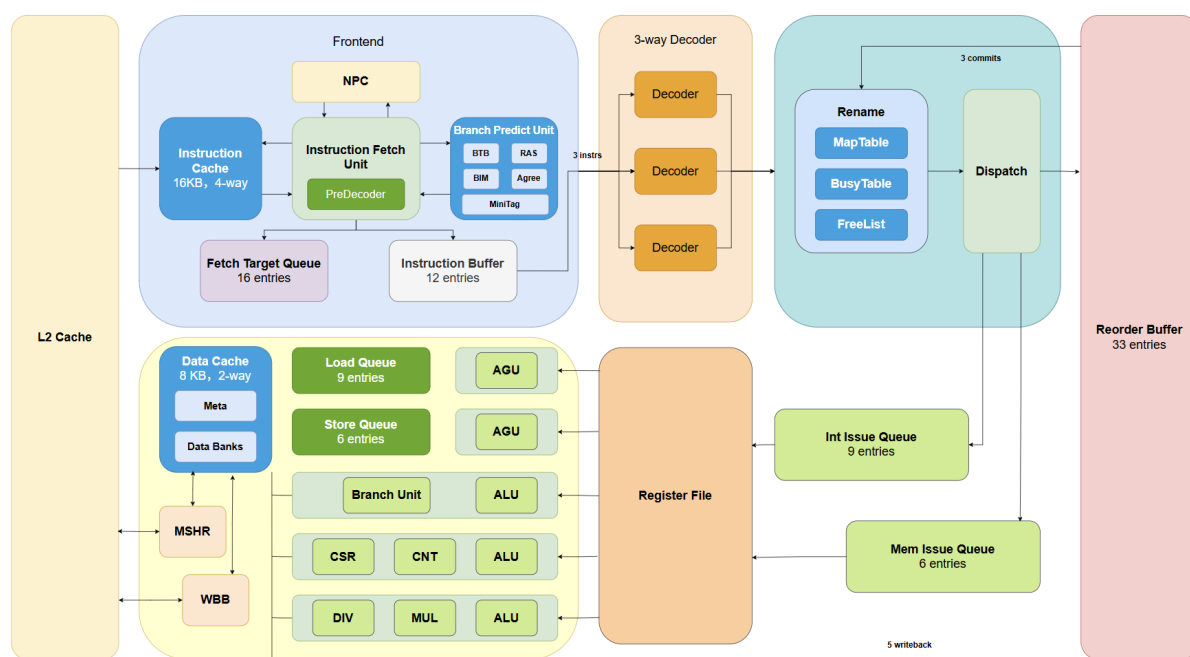


图 1.1 CPUSTCore 系统总体架构与主要数据通路

前端将预测上下文写入 FTQ，并将有效指令送入 IBuffer；乱序后端建立物理寄存器依赖，分别从整数与访存队列选择就绪指令送往五个执行端口。结果写回后由 ROB 按序提交，Cache 缺失则经 MSHR、统一 L2 和 AXI 继续处理。分支误预测与精确异常

均重定向前端，ROB 同时负责预测器训练和推测资源回收，形成完整的执行与恢复闭环。

## 1.2 实现内容概述

CPUSTCore 围绕 LA32R 指令执行组织为解耦前端、乱序后端和非阻塞访存层次；FTQ、IBuffer、发射队列和 ROB 隔开各段反压，使长延迟运算、访存重放和流水恢复只阻塞相关指令。

### 1.2.1 指令支持

核心实现 LA32R 整数指令集，覆盖整数算术、比较、逻辑、移位、乘除和取模运算，支持 B/BL/JIRL、六类条件分支以及字节、半字和字粒度的 Load/Store，并实现 LL.W/SC.W 原子访存。

特权部分实现 CSR 读写与交换、SYSCALL/BREAK/ERTN/IDLE、定时器和 CPUCFG，支持 TLB 查询、读写、填充与失效，以及 CACOP/PRELD/DBAR/IBAR。这些指令与中断、精确异常和地址翻译共同构成运行 U-Boot 与 Linux 所需的体系结构接口。

### 1.2.2 逻辑流水与模块职责

本处理器核心包含 13 个流水级，图 1.2 给出无 Cache miss、无端口冲突时的逻辑级；乘除法、访存重放、CSR 序列化和恢复事件会引入额外周期。

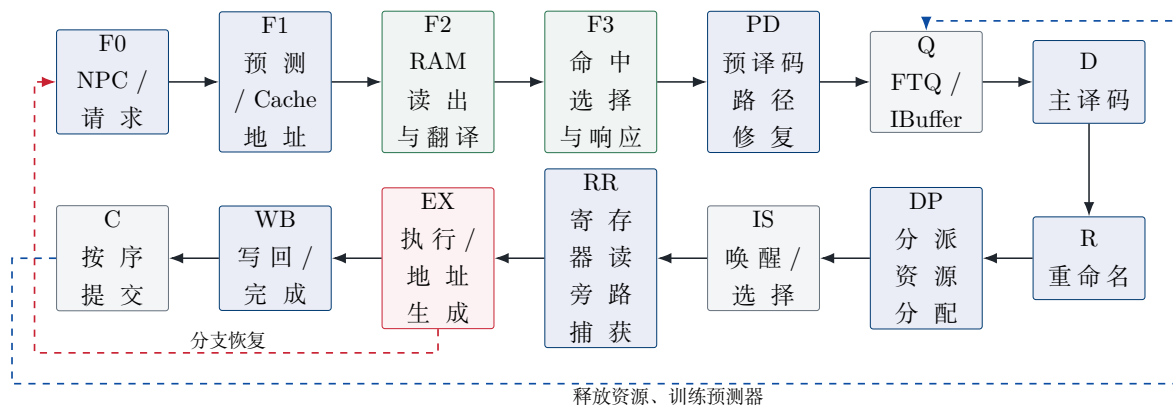


图 1.2 CPUSTCore 逻辑流水级；Q 表示可停顿队列边界

## 1.3 关键设计

### 1.3.1 多 bank 分组并行

**LVT PRF / Load relay** 多写 bank 按读口复制，LVT 指示最新值所在 bank；Load 结果先进入局部寄存器，下一拍写 PRF，同时向 RegisterRead 提供 relay。以规则的一读一写存储体实现多端口 PRF，并隔离 DCache 返回到复制体的高扇出路径。

**Rename / FreeQueue** 三宽组内以最新映射旁路消解 RAW/WAW；分配新映射时保存旧映射，银行化空闲队列维护推测头、体系结构头和分支快照。新映射可立即建立依赖，旧映射只在提交后回收；恢复只需回到已保存的资源位置。

**LoadQueue** 使用预分配 Load index 对 bank 数的余数进行 bank 选择，以最大吞吐量为代价清除交叉反馈逻辑，降低访存子系统的复杂度。

**Folded cache** Folded cache 通过将缓存按 {way, set} 的索引形式进行组织，成为多个较小的 bank 并进行折叠，极大减少了面积和资源开销 (-70%)，提高了 bram tile 深度的利用率。

### 1.3.2 非阻塞数据流

**Store Queue** 暂存 Store Queue 暂存已经提交的 store。当 DCache 暂时不可接受时，队列保持请求有效，避免 ROB 提交路径直接被 store 执行握手阻塞。

**LSAllocator** 提前管理容量 LSAllocator 提前分配并管理 LST/SQ 访存容量，自然地维护了访存子系统中指令程序序的同时，通过前置分配避免了因临时的下游回压导致阻塞和长握手链。

**非阻塞数据缓存** MSHR 收集并合并 cache miss，将缺失请求与 refill 返回过程解耦；另设置 Load State Table 实际上兼任 Load Replay Queue，使 miss load 可以在不阻塞访存流水线的情况下被打回，并等待重新发射。因此端口冲突、容量变化或 miss 延迟只限制对应缓冲点的接收，不沿整条访存路径回传形成全局停顿。

### 1.3.3 早响应机制

**缩短控制流损失** Agree 提供低延迟方向结果，MiniTAGE 利用多段全局历史补充复杂相关模式，预译码再校验控制流类型、位置和直接目标，构成预测、覆盖和提前纠正三级路径。此外，到达执行级后，BranchUnit 立即校验分支结果，并进行 PC 重定向及错误指令冲刷。

数据旁路与推测唤醒 与非阻塞数据流相结合，多数整数指令可以借助旁路解除数据依赖连续发射。而对于 Load 依赖的指令，可以对 Load 进行 hit 推测并提早唤醒对应指令，并借助 cancel 机制无 flush 撤销唤醒

**Load 流水压缩** Load 无疑是位于程序依赖链的顶端。基于此，我们合并了访存侧流水线的 RegFile 和 preAgu，为 new Load 提供 fast dcache path，为 miss load 设计 mshr fast writeback path，减少访存延迟对依赖指令的影响。

## CHAPTER 2

## 混合粒度分支预测

## 2.1 三级预测结构

分支预测由 Agree 快预测、MiniTAGE 长历史预测和预译码确认纠正组成。BTB 与 RAS 提供目标，Agree 维持低延迟方向预测；MiniTAGE 包含 4 张 512 项表，分别使用 4、12、32 和 64 位全局历史，provider 命中后覆盖 Agree 方向。

取指请求先进入 BTB/Agree 快路径，下一拍即可选择早期下一 PC。MiniTAGE 从寄存后的请求查询同步表，在 ICache 返回时与指令包对齐；预译码采用最终方向并核对槽位、类型和目标。若结果改变控制流，仅撤销较年轻的取指请求，因此长历史查询不会进入快路径关键时序。

## 2.1.1 方向选择

对条件分支槽  $i$ ，记 Agree 快路径方向为  $F_i$ ，MiniTAGE 方向为  $M_i$ ，有效 provider 命中为  $H_i$ ，则

$$P_i = \begin{cases} M_i, & H_i, \\ F_i, & \text{其他情况}. \end{cases}$$

MiniTAGE 优先使用命中的最长历史表；弱新项可采用较短历史的备选表，不存在备选命中时回退 Agree。

## 2.1.2 目标选择

BTB 完整标签命中的槽位才可提供早期目标；普通分支和间接跳转使用 BTB 目标，预测为返回指令时改用 RAS 栈顶。若一个取指块内存在多个 taken 候选，则选择程序顺序最早的槽位重定向 NPC，后续槽位不再进入有效路径。

预译码得到真实指令类型后再次选择目标。记槽位有效掩码为  $A_i$ ，控制流类型为  $Q_i$ ，最终条件方向为  $P_i$ ，BPU 目标有效位为  $V_i$ ，则重定向候选为

$$R_i = A_i \wedge (Q_i = \text{BL} \vee (Q_i = \text{JIRL} \wedge V_i) \vee (Q_i = \text{BR} \wedge P_i)).$$

令  $k = \min\{i \mid R_i\}$ 。直接跳转和条件分支采用预译码计算的 PC 相对目标；JIRL 无法仅由指令确定运行时目标，因此采用 BTB 目标，返回指令对应的值已由 RAS 覆盖。预

译码若发现槽位或目标与早期预测不一致，则在下一拍重定向；若早期 BTB 命中实际不是控制流指令，则回到顺序地址  $PC + 16$ 。JIRL 没有有效目标时不在预译码级重定向，由执行级得到真实目标后恢复。

## 2.2 恢复与训练

前端每次处理一个 16 字节、四指令取指块，BPU 与 ICache 并行查询，并以程序顺序最早的有效控制流决定块出口。GHR 保存推测方向历史，FTQ 保存入口历史、预测结果和训练快照。

预译码发现不一致时重建历史并重定向；执行级再次确认真实方向和目标。预测器只使用已退休分支按序训练，避免错误路径污染长期状态。

## 2.3 适用场景

Agree 适合强偏置、短循环和表项别名明显的分支；MiniTAGE 适合嵌套条件、跨迭代和多尺度相关。预译码补充发现 BTB 未命中、类型错误和目标错误。对于近似随机的方向或纯目标预测问题，长历史预测不会带来稳定收益。

## 2.4 消融结果

实验使用 20 个 `nscsc_perf` 测试，并保持总线随机种子、结束 PC 和 Verilator 选项一致。只改变预测结构，对比 Local、GShare 历史对照 [?]、Agree[6] 和 MiniTAGE[?]。正确率采用微平均；逐项图以 Local 正确率为  $1.0\times$ ，柱高为对应配置正确率与 Local 正确率之比。

表 2.1 预测正确率与误预测频率（20 项测试）

| 配置       | 总分支正确率   | 误预测/千分支 | 条件分支正确率  | 误预测/千条件分支 |
|----------|----------|---------|----------|-----------|
| Local    | 94.6011% | 53.989  | 91.9434% | 80.566    |
| GShare   | 96.3776% | 36.224  | 94.6096% | 53.904    |
| Agree    | 97.0630% | 29.370  | 95.6331% | 43.669    |
| MiniTAGE | 98.2318% | 17.682  | 97.3891% | 26.109    |

微平均结果显示，Agree 将总分支误预测频率从 Local 的 53.989 次/千分支降至 29.370，条件分支则从 80.566 次/千条件分支降至 43.669；MiniTAGE 进一步降至 17.682 和 26.109。逐项正确率比例中，CoreMark、quick\_sort 和 Dhrystone 分别达到  $1.101\times$ 、

1.097 $\times$  和 1.161 $\times$ ，说明收益主要集中在具有明显历史相关性的负载。

部分测试的比例接近 1.0 $\times$ ，原因是 Local 已能稳定预测方向固定的循环回边，而非绘图精度不足。例如 `inner_product`、`loop_induction` 和 `my_memcmp` 的 Local 误预测率分别只有 0.0157%、0.0081% 和 0.0095%。因此，图 2.1 用于观察整体相对变化，细小差异仍以表中的原始误预测频率为准。

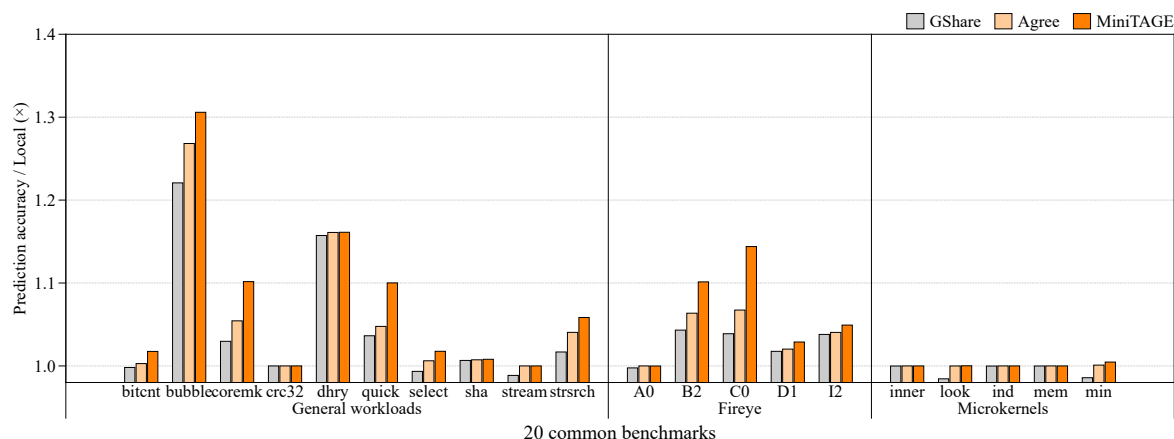


图 2.1 各配置相对 Local 的逐项预测正确率比例 (Local 为 1.0 $\times$ )

评价上述消融结果时，需要同时考察误预测率、恢复周期和 IPC。访存行为只有在改变控制流序列时，才会影响方向预测结果。

### 2.4.1 MiniTAGE 覆盖效果

图 2.2 单独统计 MiniTAGE 相对 Agree 的覆盖效果：Recover 向上表示纠正 Agree 的次数，Harm 向下表示覆盖后新增的错误， $Net = Recover - |Harm|$ ，单位均为每千条条件分支。C0、quick\_sort 和 CoreMark 的净收益最明显，说明长历史能够补充 Agree 无法表达的相关性；A0 出现轻微负收益，其余简单循环测试接近零。整体上，MiniTAGE 的纠正量明显高于新增错误量。

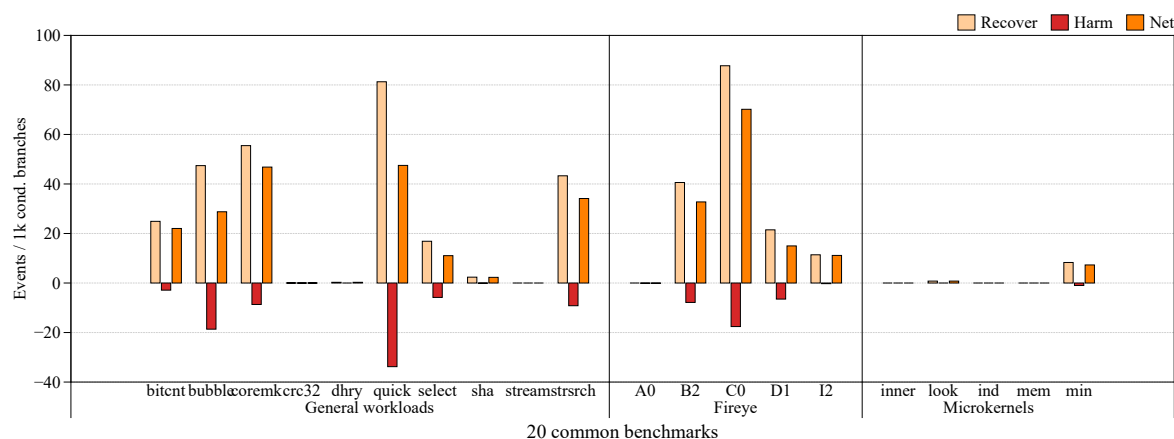


图 2.2 MiniTAGE 相对 Agree 的纠正量、伤害量与净收益



`inner_product`、`loop_induction`、`my_memcmp` 和 `stream_copy` 的纠正量与伤害量均为零；`crc32` 的 822142 次分支中仅纠正 8 次、引入错误 5 次，净收益为 3 次。这些结果说明 MiniTAGE 只在存在可学习历史关系时有效，不会普遍提高所有测试。

## CHAPTER 3

## 设计空间探索

## 3.1 实验协议

为了刻画当前架构的性能边界并评估各项策略的贡献，我们设计并可视化了一系列定量实验，在资源和时序约束下系统探索设计空间，寻找综合性能更优的架构配置。

与性能测试采用相同口径。对于配置  $c$  和负载  $b$ ，性能定义为固定输入下的完成周期加速比

$$S_{c,b} = \frac{C_{\text{baseline},b}}{C_{c,b}}, \quad S_c = \exp \left( \frac{1}{N} \sum_{b=1}^N \ln S_{c,b} \right).$$

## 3.2 机制消融

机制矩阵分别关闭 MiniTAGE override、LVT PRF 和 Load 推测唤醒，并设置三者同时关闭的组合点。图 3.1 左侧汇总 7 项负载完成周期的几何平均变化，右侧保留逐负载分布；图 3.2 基于基线配置的直接计数器结果，将 MiniTAGE 的 Recover/Harm 以及推测唤醒的 Success/Cancel 按计数器覆盖区间内每百万条提交指令进行归一化。



图 3.1 三项机制单独及联合关闭的 7 项性能影响；正值表示相对基线加速

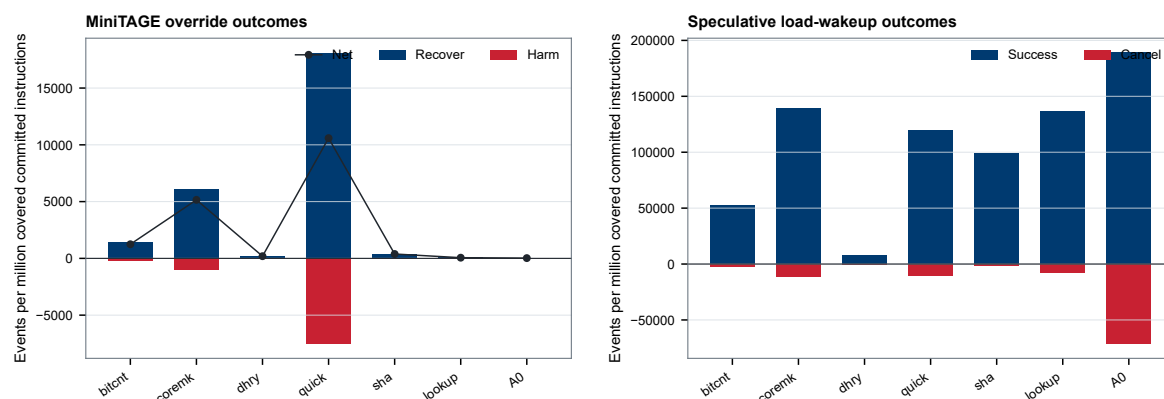


图 3.2 基线中的机制直接事件；Harm 与 Cancel 置于零线下方，所有事件按计数器覆盖区间内每百万提交指令归一化

表 3.1 机制消融性能结果

| 配置                   | $S_c$    | LUT    | FF     | BRAM | DSP |
|----------------------|----------|--------|--------|------|-----|
| 基线                   | 1.000000 | 61 488 | 31 019 | 53.5 | 4   |
| MiniTAGE override 关闭 | 0.989370 | 57 396 | 27 865 | 43.0 | 4   |
| LVT PRF 关闭           | 1.000000 | 69 380 | 32 855 | 53.5 | 4   |
| Load 推测唤醒关闭          | 0.993825 | 60 592 | 30 899 | 53.5 | 4   |
| 三项联合关闭               | 0.983003 | 65 327 | 29 602 | 43.0 | 4   |

消融结果表明，Load 推测唤醒和 MiniTAGE override 在仅增加少量资源的情况下即可带来较明显的性能收益；LVT PRF 则能够显著降低资源开销。这三项机制分别作用于控制流覆盖、物理寄存器存储组织和访存依赖唤醒，覆盖了前端预测、数据通路和访存调度三个不同的关键环节；它们共同在拥有更低的资源消耗和更好的性能同时，缩短了从预测到提交的关键路径。因此，后续参数扫描均以保留三项机制的配置作为基线。

事实上，MiniTAGE 和 Load 推测唤醒分别作为“误预测早纠正”和“访存依赖早唤醒”的机制，作为提早响应的两条路径，能够降低主流水线由于乱序带来的停顿损失，从而提高整体性能。

### 3.3 访存超参数扫描

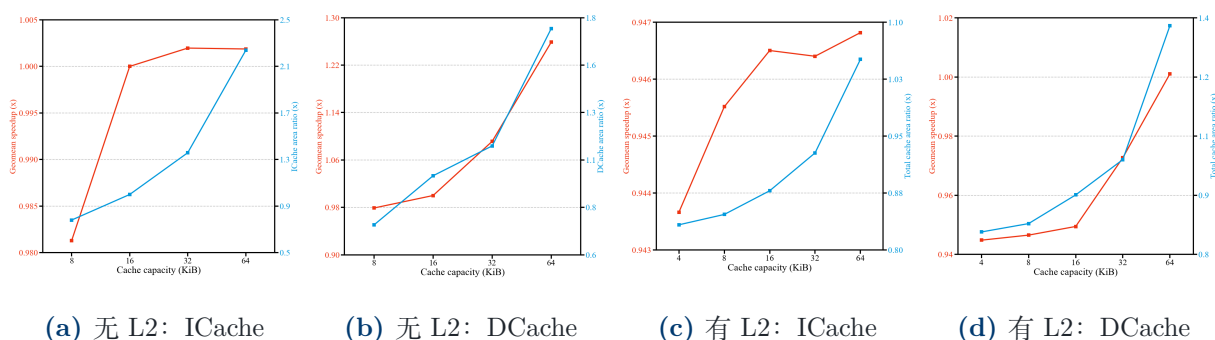


图 3.3 Cache 容量扫描。左两图关闭 L2；右两图开启固定 64 KiB L2，并以无 L2 的 I16+D64 配置归一化。

Cache 容量扫描对应的是“局部命中率”和“远端访问代价”的分工，而不是单纯追求更大的片上存储。L2 的性能提升效果在于，以脱离主流流水线处相对简易的处理逻辑，使用较低的成本，对 L1 cache miss 的访存指令进行拦截。这使得 L2 cache 在访存密集的负载中，能够有效地降低访存延迟，从而提高整体性能。如 Fireeye A0 测试项，加入 Unified L2 后，IPC up 达到了惊人的约 10 倍！不过本赛事的性能测试负载均较小且连续执行 10 轮无缓存清除，64KB 足以完全装下所有访存请求，evict 数量为 0，对驱逐后重新访问的访存行为拦截率到达 100%，故使用大规模 L1 可以起到完全相同的作用，甚至因为无 refill 效果更好。在此前提下，L2cache 的优势并不能完全展现，相比 L1 的优势仅在于统一存储为 icache 提供的便利以及布线时序上的优势。

不过在 Linux 系统启动过程中，64kL2+i8+d8 的配置相比 i32+d64，面对真实的复杂访存情况，L2 由于提供了更大的自由度，即使总面积更小，依然表现出远优于 L1 的执行速度（约 1.7x）。

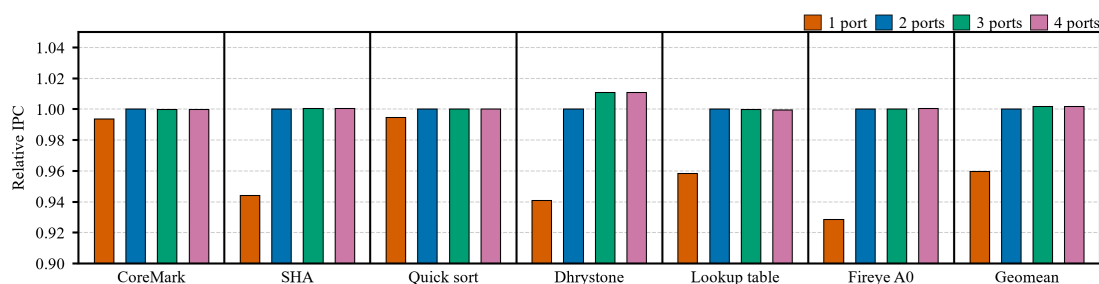


图 3.4 DCache 访问口数量扫描的相对 IPC，2-port 配置归一化为 1.0。

双路访存对单路访存在性能上有显著改善，在访存密集类测试如 Fireeye A0，以及有短窗口大量访存的场景如 SHA 中，两者差距尤为明显。尽管如此，增加 dcache 的处理端口数到 3、4 与双端口相比，在全测试集中提升均小于 1%。对测试负载而言，多数

场景下访存指令的访存系统内并行度约等于 2。事实上，在双端口情况下，Load 的平均延迟仅为约 2.1 cycles。

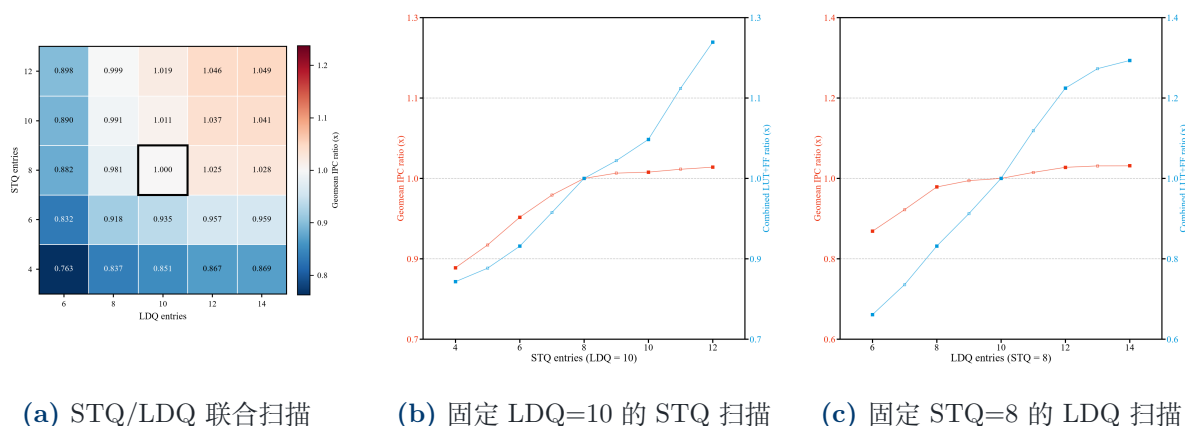


图 3.5 STQ 与 LDQ 容量扫描，均相对 STQ8+LDQ10 归一化。

STQ/LDQ 扫描对应的是访存乱序窗口。在队列容量增大的过程中性能提升边界效应明显：访存窗口尤其是 stq 容量因为其队头提交的特性和 Allocator 预分配的设计，与 rob 容量有显著联动关系。在队列容量增大的过程中性能提升边界效应明显：访存窗口尤其是 stq 容量因为其队头提交的特性和 Allocator 预分配的设计，与 rob 容量有显著联动关系。

即使发射队列采用了“Load 不能越过地址未解析的老 Store 发射”的策略，SQ 容量与 LQ 容量在全测试集合上的可采用范围内，联动效应较弱。多数负载对同时大量读写的需求较低。而在部分测试集如 String Search 中，由于连续 Store 指令情况的存在，对于 SQ 容量具有一定要求。

### 3.4 核心参数配置扫描

图 3.6 统一展示 Decode/Commit、FTQ、IBuffer、PRF、ROB、IntIQ、MemIQ、MSHR 和 WBB 的参数扫描结果。每个子图均标出当前基线值；蓝色圆点表示通过 95 MHz OOC，红色叉号表示 WNS 小于零。

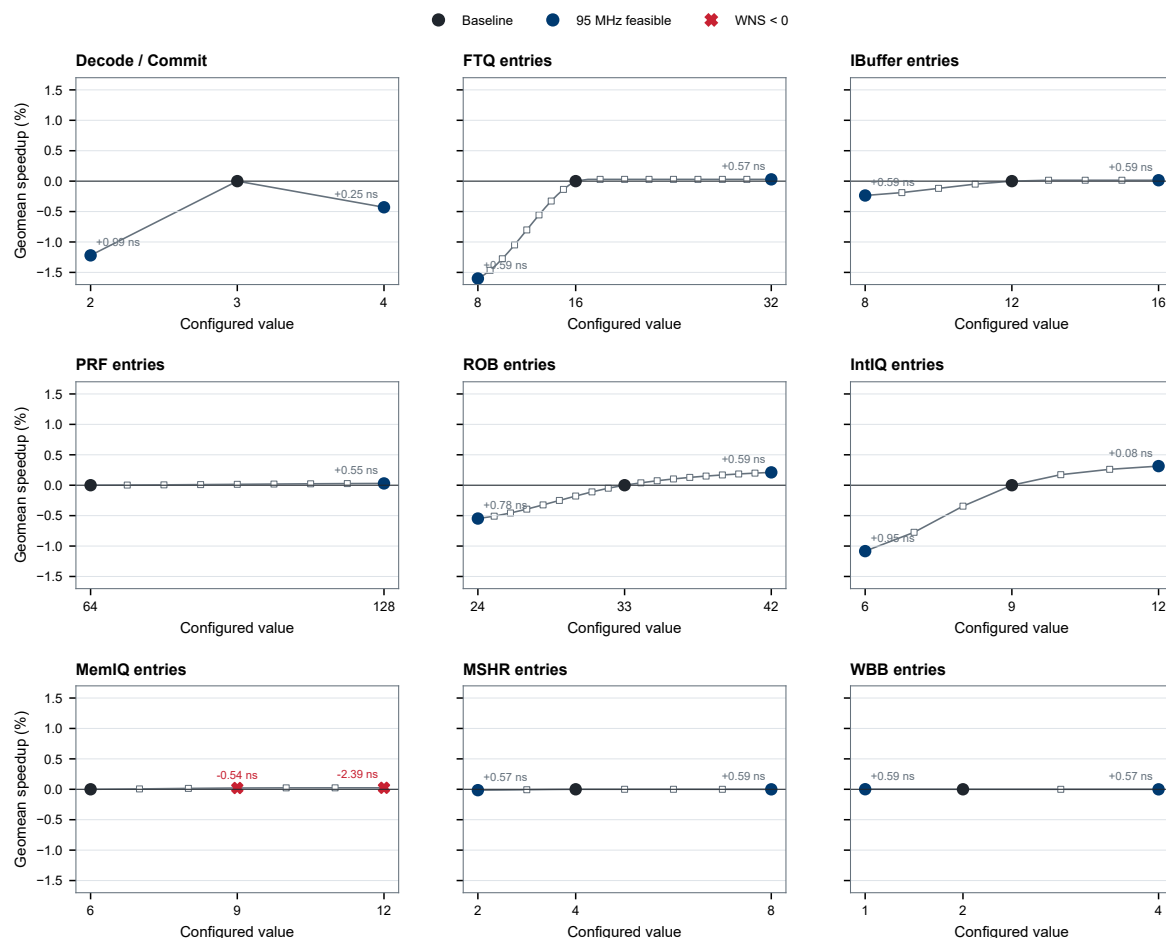


图 3.6 九组参数的性能与 95 MHz 可行性

相对于基线，各个参数的增大几乎不能带来更好的性能提升——结构参数之间存在联动效应，单独调节某个参数难免会出现核心负载不均衡的问题限制性能提高。基线本身的配置已经在保证前后端解耦的前提下，尽量限制了高代价结构的规模。值得注意的是 Decode/Commit 宽度由 3 升到 4 的性能下降。通过静态分析，我们注意到，负载指令的最大并行度大约为 3；且发射队列等由于组织形式，要求长度为发射数的倍数。从而在调节的过程中，反而使得 IntIssueQueue (9→8) 等其他机构容量有所下降，反而影响了整体性能。

### 3.5 多资源 Pareto 前沿

基于资源和性能变化进行 Pareto 前沿筛选后，图 3.7 给出 LUT 和 FF 两个资源维度上的投影。每个点的坐标分别表示相对于基线的几何平均性能和资源变化。

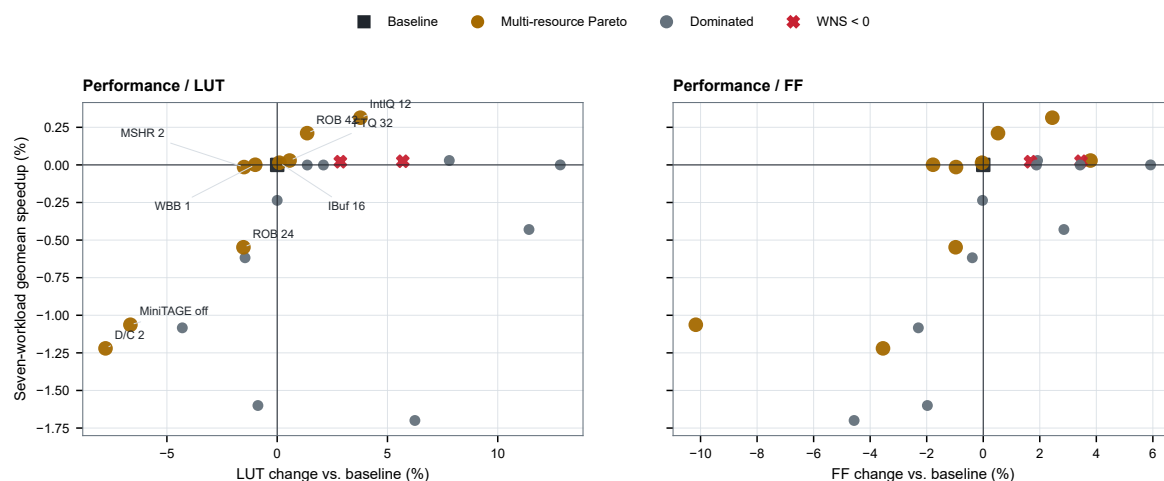


图 3.7 性能与 LUT、FF 的两个 Pareto 投影

对于 Pareto 前沿筛选得到的候选点，图 3.8 同时给出几何平均性能和全测试集逐负载变化。

由于 USTC 的代价同时来自组合逻辑和时序状态，Pareto 筛选采用 LUT 与 FF 两个资源投影。通过对图分析，我们可以发现：发射选择、依赖比较、端口仲裁等逻辑处理主要消耗 LUT；而队列项、流水寄存器、预测器状态和恢复快照等依赖状态的功能则主要消耗 FF。

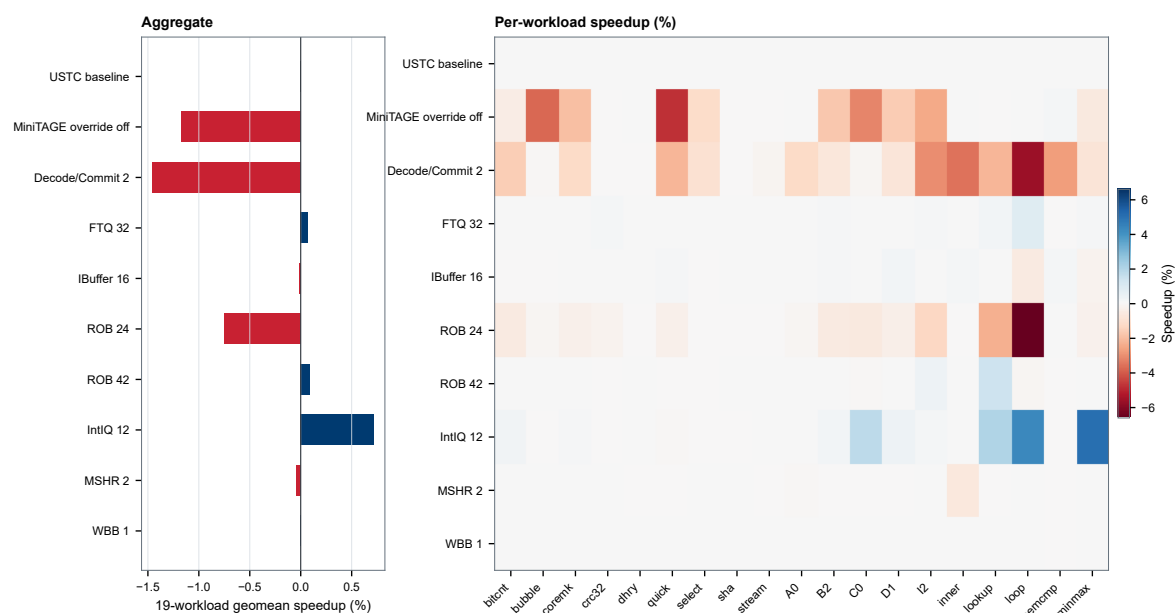


图 3.8 基线与筛选阶段 Pareto 候选的完整测试：每格为相对基线完成周期加速比变化

综合以上实验，最终版本保留前述机制；参数选择遵循“在有限规模资源和限定时序下，尽可能追求更高性能”的原则，并最终选择了如下参数：

表 3.2 USTC 当前 RTL 的主要微结构参数

| 模块   | 参数                 | 值      |
|------|--------------------|--------|
| 前端   | Decode / Commit 宽度 | 3 / 3  |
| 前端   | FTQ 深度             | 16     |
| 前端   | IBuffer 深度         | 12     |
| 执行后端 | PRF 物理寄存器数         | 64     |
| 执行后端 | ROB 深度             | 33     |
| 执行后端 | IntIQ 深度           | 9      |
| 执行后端 | MemIQ 深度           | 6      |
| 存储层次 | ICache 容量          | 16 KiB |
| 存储层次 | DCache 容量          | 8 KiB  |
| 存储层次 | 统一 L2 Cache 容量     | 64KiB  |
| 存储层次 | LST/LDQ 深度         | 12     |
| 存储层次 | SQ/STQ 深度          | 10     |
| 存储层次 | MSHR 数量            | 4      |
| 存储层次 | WBB 深度             | 2      |



## CHAPTER 4

## CPUTSC SoC 系统设计

**CPUTSC SoC** 是以 CPUTSCore 为主处理器、面向 Xilinx xc7a200t FPGA 的片上系统。它包括 DDR3、AXI/APB 互联、时钟与复位、中断、板级外设、DMA 和专用加速器，并为 U-Boot 与 Linux 提供硬件平台。板级外设包括 UART、LED、数码管、按键、SPI NOR Flash、NAND Flash、SD 卡、Ethernet、USB、VGA、LCD、Goodix 触摸屏和 OV7670 摄像头；GPU 与 TensorCore 是专用加速器。操作系统及用户程序涉及桌面、识别、摄像、存储和网络连接等，作为真实使用场景提供验证。

## 4.1 系统组成与互联存储

CPUTSCore 工作在 50 MHz。外设控制逻辑工作在 33 MHz。VGA、LCD、GPU、TensorCore、Camera、USB、Ethernet、NAND 和 SDIO 通过 AXI 访问 DDR3。CPU 负责配置寄存器、状态查询和中断处理。DMA 负责图像、张量和块数据传输。

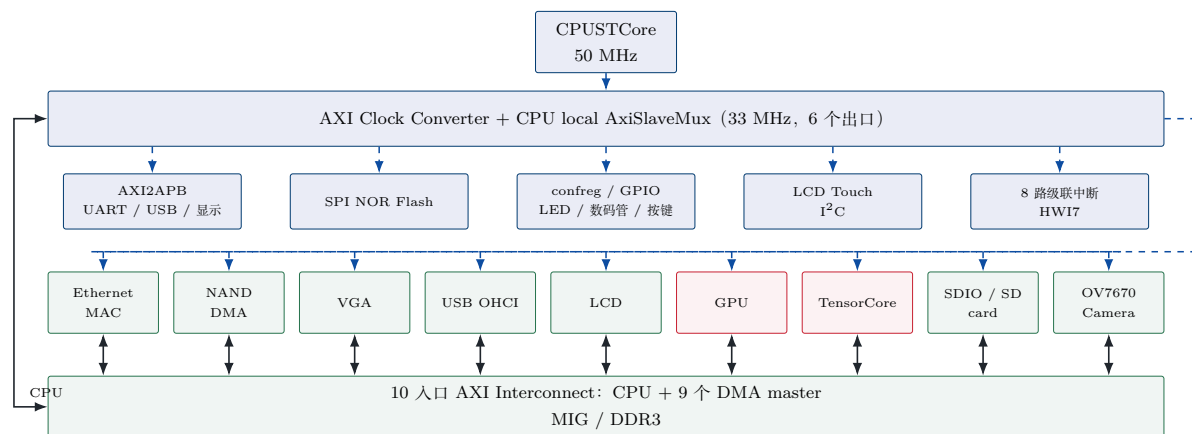


图 4.1 CPUTSC SoC 的控制面与 DDR 数据面

CPU 请求先通过 AXI Clock Converter 进入 33 MHz 外设域，再由本地 AXI 路由器分配到 DDR、SPI、APB、confreg、Ethernet 或 SDIO。APB 连接 UART、USB、VGA、GPU、TensorCore、触摸 I2C、LCD、中断控制器和 NAND。

DDR 互联包含十个入口。一个入口连接 CPU，另外九个入口连接 Ethernet、NAND、VGA、USB、LCD、GPU、TensorCore、SDIO 和 Camera。各设备使用独立 AXI 入口。

MIG 负责 DDR3 控制。这种结构减少 CPU 搬运，也允许显示、计算和 I/O 并行工作。

## 4.2 外设与加速器

### 4.2.1 显示与交互

VGA 从 DDR3 读取 RGB565 帧缓冲，并输出 640×480 画面。硬件光标使用独立图层，移动光标时不需要重画桌面。GPU 支持矩形填充、区域复制、单色图展开。LCD 控制器连接 16 位 8080 接口，并通过 DMA 读取帧缓冲。触摸控制器通过 I<sup>2</sup>C 接入系统。

USB OHCI 连接 USB3500 PHY，可接入键盘和鼠标。

OV7670 摄像头输出 640×480 RGB565 图像。采集前端使用 8 KiB 异步 FIFO 完成跨时钟域传输，再通过 AXI DMA 写入 DDR3。传感器配置使用 SCCB 接口。

### 4.2.2 存储与网络

SPI NOR Flash 装载 U-boot 用于启动。NAND 一般用于存储 Linux 镜像。

SDIO 利用龙芯开发板右侧扩展接口接入，支持 4 Pin 传输模式，并通过 DMA 访问 DDR3。SDIO DMA 支持 SG (Scatter-Gather)。控制器预取描述符表后，可按多个物理不连续的缓冲区连续传输数据，并兼容原有单段 DMA。该功能使得小块数据的 DMA 传输连续进行，减少 CPU 参与，大大提高吞吐率。

SD 卡测试完成了分区识别、文件系统挂载、写入、同步和回读校验。批量读取吞吐率为 13.70 MiB/s，写入吞吐率为 4.67 MiB/s。在同一镜像下，由于采取缓存一致性维护和 DMA 传输的并行化策略，双流读取比 10.68 MiB/s 的单流读取提高 28.3%。

将开发板连接 CLK 和 GND 的线绞缠在一起，解决了 SDIO 接线的串扰问题，使得 SD 卡在板端可以较稳定工作。

USB OHCI 连接 USB3500 PHY，可接入存储设备和无线网卡。USB 主机完成了存储设备枚举，支持 U 盘挂载及读写。无线网卡通过 USB 完成模式切换、驱动绑定和 WPA/WPA2 关联，并实测拉取了真实网页、完成了图片传输。

### 4.2.3 TensorCore

TensorCore 使用  $1 \times 4$  FP32 阵列，核心计算单元由抢占科技制高点专项课题提供支持，在 SoC 上扩展为了通用矩阵乘法单元。CPU 通过 APB 配置任务，TensorCore 通过 AXI DMA 读取输入并写回结果。完成和错误状态通过中断报告。矩阵数据采用 FP32 row-major 格式。该单元用于全连接层和卷积计算，并支持批量矩阵乘法、卷积窗口生成、PReLU 和最大池化。最新 ABI 支持以工作区保存层间中间张量，用户态仅提

交任务描述和工作区，减少 CPU 重复数据搬运与寄存器配置。

## 4.3 软件与应用

### 4.3.1 启动与软件层次

系统启动分为引导和操作系统两个阶段。上电后，U-Boot 完成必要的硬件初始化，支持从网线传输 Linux 镜像并固化到 NAND。复位后实现 autoboot，从 NAND 加载 Linux 内核，并把系统控制权交给内核。Linux 随后初始化内存、中断和设备驱动，挂载由 Buildroot 生成的根文件系统。根文件系统包含基础命令、运行库、Xorg、Matchbox、图形库和应用程序。

设备驱动运行在 Linux 内核中，负责配置外设寄存器、分配 DMA 缓冲区、处理中断，并保持 CPU 缓存与 DMA 数据一致。Xorg 负责显示输出和输入事件，Matchbox 负责窗口管理，图形库向应用提供绘制接口。应用运行在用户态，不直接访问硬件寄存器：图形程序通过图形库和 Xorg 使用显示设备。

### 4.3.2 桌面与帧处理

桌面使用 RGB565 共享缓冲。GPU 处理大面积复制，硬件光标处理鼠标移动，大大降低 CPU 负担。对已渲染页面采用 LRU 缓存，并保留背景缓存；新渲染桌面页面时可直接 GPU 复制缓存背景，避免重复绘制。在板端测试中，冷打开桌面页面耗时平均约 800ms，热页面切换平均耗时为 110.893 ms；早期基线为 2497.234 ms，耗时下降 95.56%，加速 22.52 $\times$ 。两组结果来自不同阶段的软件镜像，用于说明整体演进。

### 4.3.3 手写数字识别与触摸

我们实现了有机器学习的“Hello World”之称的手写数字识别应用，并嵌入到真实交互场景中：在板端运行小型神经网络识别数字，给用户实时反馈。

用户在 LCD 触摸屏写入数字，应用在 480 $\times$ 800 LCD 的 440 $\times$ 440 区域接收笔画，CPU 完成图像裁剪、缩放和结果处理，TensorCore 完成 256-64-32-10 多层感知机的 27 次矩阵乘法，最终将识别结果显示在屏幕上。MNIST 10,000 张测试图像的离线准确率为 97.44%。

### 4.3.4 Camera 与人脸检测

人脸检测应用实现了摄像头采集、即时检测人脸的流程，并支持检测人脸的情绪，展现了 CPUSTC SoC 的实时处理能力。

Linux 将摄像头采集缓冲作为 DMA-BUF 交给 Xorg，三个共享缓冲分别用于采集、

等待显示和屏幕扫描。程序每秒取当前帧做人脸检测，检测结果将在计算成功后立即更新到屏幕上，显示为矩形人脸框。板端在开启人脸检测情况下实际帧率为 3.210 fps，达到 5 fps 配置值的 64.2%。在仅显示视频流的情况下，最大帧率达到 5-6 fps。

P-Net 使用四级图像金字塔，每次检测执行 20 次 TensorCore 计算。检测线程延迟的 P50/P90 为 277.806/388.341 ms，端到端延迟的 P50/P90 为 384.129/624.230 ms，检测结果更新率为 0.869 次/s。端到端 P50 相比 5.381 s 的早期基线下降 92.9%，加速 14.01 $\times$ 。

检测到人脸后，程序选取置信度最高的人脸区域，裁剪并缩放为 48 $\times$ 48 灰度图。情绪识别网络通过 TensorCore 执行四层卷积和一层全连接，输出中性、开心、惊讶、悲伤、愤怒等八类结果及置信度。界面提供独立的 Face 和 Emotion 开关；关闭 Emotion 仅停止情绪识别，仍保留人脸框检测。

相机快照还可通过 Wi-Fi 传输到手机。板端连接手机热点后，执行脚本获取访问地址；手机浏览器打开该地址即可查看或下载拍摄图像。图像由手机浏览器完成显示转换，板端直接传输相机保存的原始图像数据。

### 4.3.5 板端编译环境

系统移植了面向 LoongArch32 软浮点 ABI 的 GCC 8.3，本地工具链以 ext4 镜像部署在 SD 卡。板端已经实测通过基于 xterm 和 vi 能正常编译 c 程序并运行。CPython 3.11.2 以 ext4 镜像部署在 SD 卡，标准库包含 1,295 个预编译文件。板端已经实测成功运行 Python 脚本及启动 Python 解释器。

### 4.3.6 其他图形应用

桌面可运行小游戏，例如五子棋、扫雷、平台跳跃和显示在龙芯试验箱的 8\*8 LED 点阵的贪吃蛇游戏。

桌面支持 xterm 终端，用户可在 UI 界面直接使用命令行工具。

桌面还移植了轻量浏览器 Netsurf，冷加载本地欢迎页耗时 10-20 s，滚动页面时，GPU 复制上一帧已显示区域，CPU 渲染新增区域，耗时平均约为 1.5 s。上板实测，欢迎页三次滚动刷新字体光栅化耗时占 94.76%。同时已经通过无线网卡连接手机热点，实测访问了中国科学技术大学官方网站首页，但渲染时长达几分钟。浏览器的渲染效率可作为后续改进方向。

CHAPTER 5

实验结果

5.1 性能与时序摘要

表 5.1 决赛正式性能与实现结果

| 指标                      | 结果               |
|-------------------------|------------------|
| 提交目标频率                  | 95 MHz           |
| 实现后实际频率                 | 94.73684 MHz     |
| IPC 相对参考核比值 (20 项几何平均)  | 2.429190206×     |
| 系统计数器相对参考核比值 (20 项几何平均) | 7.042610727×     |
| Setup WNS / TNS         | +0.196 ns / 0 ns |
| Hold WNS / TNS          | +0.052 ns / 0 ns |

5.2 性能测试详细结果

表 5.2 决赛正式性能评测逐项计分数据

| 测试程序         | 采用运行 | 本核 CPU 计数 | 参考核 CPU 计数 | $R_i$    | $S_i$    |
|--------------|------|-----------|------------|----------|----------|
| bitcount     | 1    | 158,644   | 343,640    | 2.166108 | 6.275388 |
| bubble_sort  | 1    | 835,375   | 1,599,254  | 1.914414 | 5.543291 |
| coremark     | 1    | 1,989,165 | 4,130,786  | 2.076643 | 6.012766 |
| crc32        | 2    | 1,736,806 | 2,583,293  | 1.487381 | 4.316440 |
| dhystone     | 2    | 25,478    | 46,601     | 1.829068 | 5.330543 |
| quick_sort   | 1    | 1,291,721 | 1,969,534  | 1.524736 | 4.415211 |
| select_sort  | 2    | 472,090   | 1,041,006  | 2.205101 | 6.385591 |
| sha          | 1    | 1,248,999 | 2,190,475  | 1.753784 | 5.093731 |
| stream_copy  | 1    | 78,279    | 130,647    | 1.668992 | 4.853645 |
| stringsearch | 2    | 518,337   | 581,771    | 1.122380 | 3.285882 |

| 测试程序                | 采用运<br>行 | 本核 CPU 计<br>数 | 参考核 CPU<br>计数 | $R_i$     | $S_i$     |
|---------------------|----------|---------------|---------------|-----------|-----------|
| fireye_A0           | 2        | 415,052       | 4,240,749     | 10.217392 | 29.504189 |
| fireye_B2           | 2        | 231,349       | 545,430       | 2.357607  | 6.829265  |
| fireye_C0           | 1        | 390,076       | 676,520       | 1.734329  | 5.024298  |
| fireye_D1           | 2        | 1,567,001     | 6,038,605     | 3.853606  | 11.156142 |
| fireye_I2           | 2        | 1,989,404     | 3,906,531     | 1.963669  | 5.685140  |
| inner_product       | 1        | 3,376,511     | 16,232,737    | 4.807547  | 13.917781 |
| lookup_table        | 2        | 890,910       | 2,958,023     | 3.320227  | 9.614340  |
| loop_induction      | 1        | 2,810,860     | 11,084,756    | 3.943546  | 11.416205 |
| my_memcmp           | 1        | 871,194       | 3,395,161     | 3.897135  | 11.282426 |
| minmax_<br>sequence | 2        | 1,545,621     | 3,942,826     | 2.550966  | 7.385560  |

逐项结果中, fireye\_A0、inner\_product、loop\_induction 和 my\_memcmp 等项目的核心计数器比值较高, 说明其工作集和控制流更容易从乱序窗口、整数发射和预测器覆盖中获益; stringsearch、crc32 等项目的提升相对有限, 则说明其瓶颈更可能落在小窗口爆发访存等待、数据依赖、外部访问路径。



## 参考文献

- 
- [1] NOP 团队. *NOP: A Strong Baseline for LoongArch Out-of-order Processor*[R]. 全国大学生计算机系统能力大赛 CPU 设计赛决赛报告, 2023.
  - [2] Loongson Technology. *LoongArch Reference Manual, Volume 1: Basic Architecture*[EB/OL]. <https://loongson.github.io/LoongArch-Documentation/LoongArch-Vol1-EN.html>.
  - [3] 姚永斌. 超标量处理器设计 [M]. 北京: 清华大学出版社, 2014. ISBN 978-7-302-34707-1.
  - [4] 毛德操. *RISC-V CPU 芯片设计: 香山源代码剖析* [M]. 杭州: 浙江大学出版社, 2024. ISBN 978-7-308-24959-1.
  - [5] S. Palacharla, N. P. Jouppi, and J. E. Smith. *Complexity-Effective Superscalar Processors*[C]// *Proceedings of the 24th Annual International Symposium on Computer Architecture*. New York: ACM, 1997: 206–218. <https://doi.org/10.1145/264107.264201>.
  - [6] E. Sprangle, R. S. Chappell, M. Alsup, and Y. N. Patt. *The Agree Predictor: A Mechanism for Reducing Negative Branch History Interference*[C]// *Proceedings of the 24th Annual International Symposium on Computer Architecture*. New York: ACM, 1997: 284–291. <https://doi.org/10.1145/264107.264210>.
  - [7] A. Moshovos, S. E. Breach, T. N. Vijaykumar, and G. S. Sohi. *Dynamic Speculation and Synchronization of Data Dependences*[C]// *Proceedings of the 24th Annual International Symposium on Computer Architecture*. New York: ACM, 1997: 181–193. <https://doi.org/10.1145/264107.264189>.
  - [8] G. Z. Chrysos and J. S. Emer. *Memory Dependence Prediction Using Store Sets*[C]// *Proceedings of the 25th Annual International Symposium on Computer Architecture*. Los Alamitos: IEEE Computer Society, 1998: 142–153. <https://doi.org/10.1109/ISCA.1998.694770>.
  - [9] S. Mashimo, K. Inoue, R. Shioya, A. Fujita, R. Matsuo, S. Akaki, A. Fukuda, T. Koizumi, J. Kadomoto, H. Irie, and M. Goshima. *An Open Source FPGA-Optimized Out-of-Order RISC-V Soft Processor*[C]// *2019 International Conference on Field-Programmable Technology*. Piscataway: IEEE, 2019: 63–71. <https://doi.org/10.1109/ICFPT47387.2019.00016>.
  - [10] H. Wong, V. Betz, and J. Rose. *Microarchitecture and Circuits for a 200 MHz Out-of-Order Soft Processor Memory System*[J]. *ACM Transactions on Reconfigurable Technology and Systems*, 2017, 10(1): Article 2, 1–22. <https://doi.org/10.1145/2974022>.

- 
- [11] The BOOM Project. *Reorder Buffer (ROB) Documentation*[EB/OL]. <https://docs.boom-core.org/en/latest/sections/reorder-buffer.html>.
- [12] XiangShan Project. *XiangShan: Open-source High-performance RISC-V Processor*[CP/OL]. <https://github.com/OpenXiangShan/XiangShan>.
- [13] iFuProcessor. *iFuCore*[CP/OL]. 2024[2026-08-03]. <https://github.com/iFuProcessor/iFuCore>.
- [14] MAdrid1011. *Zircon-2024: A RISC-V Processor*[CP/OL]. 2024[2026-08-03]. <https://github.com/MAdrid1011/Zircon-2024>.
- [15] AMD/Xilinx. *UltraFast Design Methodology Guide for the Vivado Design Suite, UG949*[EB/OL].
- [16] 抢占科技制高点专项课题。